

Practical – 7

Government College of Engineering, Jalgaon (An Autonomous Institute of Govt. of Maharashtra) Department of Computer Engineering	
Name:	
PRN:	Batch:
Class:	Semester:
Course:	Subject Teacher:
Date of Performance:	Date of Completion:

Aim : Program for Distributed chat server using TCP sockets.

Title: Design a distributed Program for Distributed chat server using TCP sockets.

Software Requirements: Ubuntu OS, Eclipse IDE 4.11

Theory :

Client/Server Communication

At a basic level, network-based systems consist of a server, client, and a media for communication as shown in Fig. 1.1. A computer running a program that makes a request for services is called client machine. A computer running a program that offers requested services from one or more clients is called server machine. The media for communication can be wired or wireless network.

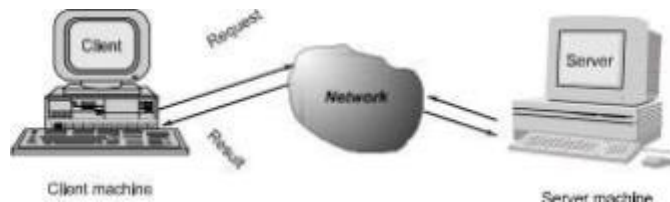


Fig.1.1 Client – Server communication

The client-server model is a distributed communication framework of network processes among service requestors, clients and service providers. The client-server connection is established through a network or the Internet.

The client-server model is a core network computing concept also building functionality for email exchange and Web/ database access. Web technologies and protocols built around the client-server model are:

- Hypertext Transfer Protocol (HTTP)
- Domain Name System (DNS)
- Simple Mail Transfer Protocol (SMTP)
- Telnet

Clients include Web browsers, chat applications, and email software, among others. Servers include

Web, database, application, chat and email, etc.

A server manages most processes and stores all data. A client requests specified data or processes.

The server relays process output to the client. Clients sometimes handle processing, but require server data resources for completion.

The client-server model differs from a peer-to-peer (P2P) model where communicating systems are the client or server, each with equal status and responsibilities. The P2P model is decentralized

networking. The client-server model is centralized networking.

One client-server model drawback is having too many client requests underrun a server and lead to improper functioning or total shutdown. Hackers often use such tactics to terminate specific organizational services through distributed denial-of-service (DDoS) attacks.

Generally, programs running on client machines make requests to a program (often called as server program) running on a server machine. They involve networking services provided by the transport layer, which is part of the Internet software stack, often called TCP/IP (Transport Control Protocol/Internet Protocol) stack, shown in Fig. 1.2. The transport layer comprises two types of protocols, TCP (Transport Control Protocol) and UDP (User Datagram Protocol). The most widely used programming interfaces for these protocols are sockets.

TCP is a connection-oriented protocol that provides a reliable flow of data between two computers



Fig 1.2 TCP/IP software stack

applications that use such services are HTTP, FTP, and Telnet.

UDP is a protocol that sends independent packets of data, called datagrams, from one computer to another with no guarantees about arrival and sequencing. Example applications that use such services include Clock server and Ping. The TCP and UDP protocols use ports to map incoming data to a particular process running on a computer. Port is represented by a positive (16-bit) integer value. Some ports have been reserved to support common/well known services:

Σ FTP 21/tcp

Σ telnet 23/tcp

Σ SMTP 25/tcp

Σ Login 513/tcp

Σ http 80/tcp,udp

443/tcp,udp

User-level process/services generally use port number value ≥ 1024 . Object-Oriented Programming with Java

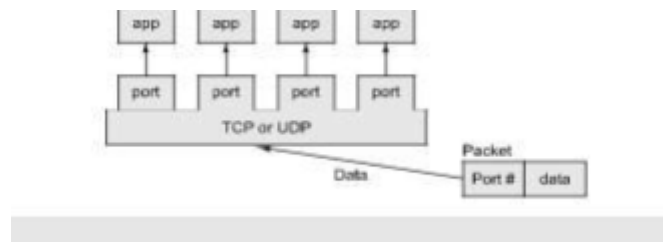


Fig. 1.3 TCP/UDP mapping of incoming packets to appropriate port/process

Object-oriented Java technologies—Sockets, threads, RMI, clustering, Web services— have emerged as leading solutions for creating portable, efficient, and maintainable large and complex

Internet applications.

ALGORITHM:

Steps:

TCP Server

Step1: Create a socket

Step2: Bind it to the operating system.

Step3: Listen over it.

Step4: Accept connections.

Step5: Receive data from client and send it back to client.

Step6: Close the socket.

TCP Client Step1:

Create a socket.

Step2: connect to the server using connect().

Step3: send data to server and receive data from the server. Step4:

Close the socket.

Conclusion:

In this practical, We learnt about the implementation of a distributed chat server using TCP sockets

Code :

Server.java

```
import socket
import threading
```

```
# Server class to handle client connections
```

```
class ChatServer:
```

```
    def __init__(self, host='127.0.0.1', port=65432):
        self.host = host
        self.port = port
        self.server_socket = None
        self.clients = [] # List to keep track of connected clients
        self.client_addresses = {} # Dictionary to store client address and socket
```

```
    def start(self):
```

```
        # Create a TCP/IP socket
        self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server_socket.bind((self.host, self.port))
        self.server_socket.listen(5) # Max number of queued connections
        print(f'Server started on {self.host}:{self.port}')
```

```
        # Accept incoming connections
```

```
        while True:
            client_socket, client_address = self.server_socket.accept()
            print(f'New connection from {client_address}')
```

```

        self.clients.append(client_socket)
        self.client_addresses[client_socket] = client_address

        # Start a new thread to handle the client
        client_thread = threading.Thread(target=self.handle_client, args=(client_socket,))
        client_thread.start()

def handle_client(self, client_socket):
    # Handle communication with a single client
    try:
        client_socket.send("Welcome to the chat server!".encode('utf-8'))

        while True:
            # Receive message from the client
            message = client_socket.recv(1024).decode('utf-8')
            if not message:
                break # Client disconnected

            # Broadcast the message to all connected clients
            print(f'Message from {self.client_addresses[client_socket]}: {message}')
            self.broadcast_message(message, client_socket)
        except Exception as e:
            print(f'Error handling client {self.client_addresses.get(client_socket)}: {e}')
        finally:
            # Cleanup when client disconnects
            self.clients.remove(client_socket)
            del self.client_addresses[client_socket]
            client_socket.close()

def broadcast_message(self, message, sender_socket):
    # Broadcast message to all clients except the sender
    for client_socket in self.clients:
        if client_socket != sender_socket:
            try:
                client_socket.send(message.encode('utf-8'))
            except Exception as e:
                print(f'Error broadcasting to {client_socket}: {e}')

if __name__ == "__main__":
    chat_server = ChatServer()
    chat_server.start()

```

Client.java

```
import socket
import threading

# Client class to interact with the chat server
class ChatClient:
    def __init__(self, host='127.0.0.1', port=65432):
        self.host = host
        self.port = port
        self.client_socket = None

    def start(self):
        # Create a TCP/IP socket and connect to the server
        self.client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.client_socket.connect((self.host, self.port))
        print(f'Connected to chat server at {self.host}:{self.port}')

        # Start a thread to listen for incoming messages
        receive_thread = threading.Thread(target=self.receive_messages)
        receive_thread.daemon = True
        receive_thread.start()

        # Main thread will handle sending messages
        self.send_messages()

    def receive_messages(self):
        # Continuously listen for messages from the server
        while True:
            message = self.client_socket.recv(1024).decode('utf-8')
            if message:
                print(f'\n{message}')

    def send_messages(self):
        # Continuously prompt the user to send messages
        while True:
            message = input("> ")
            if message:
                self.client_socket.send(message.encode('utf-8'))

if __name__ == "__main__":
    chat_client = ChatClient()
    chat_client.start()
```

Output :

Client-

```
D:\Me\College\DSL\7_TCP>python Client.py
Connected to chat server at 127.0.0.1:65432
>
Welcome to the chat server!
Helloo
> How are you
> Have a great day ahead
> See you
> Byeee !
> □
```

Server-

```
D:\Me\College\DSL\7_TCP>python Server.py
Server started on 127.0.0.1:65432
New connection from ('127.0.0.1', 51326)
Message from ('127.0.0.1', 51326): Helloo
Message from ('127.0.0.1', 51326): How are you
Message from ('127.0.0.1', 51326): Have a great day ahead
Message from ('127.0.0.1', 51326): See you
Message from ('127.0.0.1', 51326): Byeee !
□
```

Conclusion: In this practical we learnt about implementation of distributed chat application using TCP sockets.

Sign of Teacher
Mrs. Archana Chitte